

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

**Лабораторная работа №4 по курсу**

**«Операционные системы»**

Группа: М8О-211БВ-24

Студент: Отмахов Н.А.

Преподаватель: Бахарев В.Д.

Оценка: \_\_\_\_\_

Дата: 25.11.25

Москва, 2025

## Постановка задачи

### Вариант 8.

#### Функция 1:

Расчет интеграла функции  $\sin(x)$  на отрезке  $[A, B]$  с шагом  $e$ :

Сигнатура функции: `float sin_integral(float a, float b, float e);`

- Реализация №1: Подсчет интеграла методом прямоугольников;
- Реализация №2: Подсчет интеграла методом трапеций.

#### Функция 2:

Отсортировать целочисленный массив:

Сигнатура функции: `int *sort(int *array, size_t n);`

- Реализация №1: Пузырьковая сортировка;
- Реализация №2: Сортировка Хоара

## Общий метод и алгоритм решения

- `void *dlopen(const char *filename, int flags);` – загружает в память и открывает динамическую библиотеку.
- `void *dlsym(void *handle, const char *symbol);` – возвращает адрес функции или переменной из загруженной библиотеки.
- `int dlclose(void *handle);` – выгружает из памяти ранее загруженную динамическую библиотеку.

Я создал две динамические библиотеки, которые реализуют два контракта двух функций. Также я сделал два вида подключения этих библиотек в программу: на стадии компиляции и во время выполнения программы. Первый способ заключается в том, что я подключаю библиотеку во время линковки программы. Второй способ заключается в том, что я подключаю библиотеку в runtime через интерфейс ОС для работы с динамическими библиотеками.

## Код программы

### lib1.c

```
#include <math.h>
#include "../include/libfunc.h"

float sin_integral(float a, float b, float e)
{
    float sum = 0;
    for (float x = a; x < b; x += e)
    {
        sum += sinf(x) * e;
    }
    return sum;
}

int* sort(int* array, size_t n)
{
    for (size_t i = 0; i < n - 1; ++i)
    {
        for (size_t j = 0; j < n - i - 1; ++j)
        {
            if (array[j] > array[j + 1])
            {
                int tmp = array[j];
                array[j] = array[j + 1];
                array[j + 1] = tmp;
            }
        }
    }
    return array;
}
```

### lib2.c

```
#include <math.h>
#include "../include/libfunc.h"

float sin_integral(float a, float b, float e)
{
    float sum = 0;
    for (float x = a; x < b; x += e)
    {
        sum += (sinf(x) + sinf(x + e)) * e * 0.5;
    }
    return sum;
}

void quicksort(int* arr, int low, int high)
```

```

{
    if (low >= high)
    {
        return;
    }
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; ++j)
    {
        if (arr[j] <= pivot)
        {
            ++i;
            int tmp = arr[i];
            arr[i] = arr[j];
            arr[j] = tmp;
        }
    }
    arr[high] = arr[i + 1];
    arr[i + 1] = pivot;
    quicksort(arr, low, i);
    quicksort(arr, i + 2, high);
}

int* sort(int* array, size_t n)
{
    if (n > 0)
    {
        quicksort(array, 0, (int)n - 1);
    }
    return array;
}

```

## static.c

```

#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include "include/libfunc.h"

int main()
{
    char buf[512];
    while (1)
    {
        int n = read(STDIN_FILENO, buf, sizeof(buf) - 1);
        if (n <= 0)
        {
            break;
        }
        buf[n] = '\0';
        if (buf[n-1] == '\n')

```

```

{
    buf[n-1] = '\0';
}
char* args[64];
int argc = 0;
char* p = buf;
while (*p)
{
    while (*p == ' ')
    {
        p++;
    }
    if (!*p)
    {
        break;
    }
    args[argc++] = p;
    while (*p && *p != ' ')
    {
        p++;
    }
    if (*p)
    {
        *p++ = '\0';
    }
}

if (argc == 0)
{
    continue;
}
if (strcmp(args[0], "1") == 0 && argc == 4)
{
    float a = strtod(args[1], NULL);
    float b = strtod(args[2], NULL);
    float e = strtod(args[3], NULL);
    float res = sin_integral(a, b, e);
    char out[64];
    int len = snprintf(out, sizeof(out), "%.6f\n", res);
    write(STDOUT_FILENO, out, len);
}
else if (strcmp(args[0], "2") == 0 && argc >= 2)
{
    int n = (int)strtol(args[1], NULL, 10);
    if (n <= 0)
    {
        continue;
    }
    int* arr = malloc(n * sizeof(int));
    for (int i = 0; i < n && i + 2 < argc; ++i)
    {
        arr[i] = (int)strtol(args[2 + i], NULL, 10);
    }
}

```

```

    }
    sort(arr, n);
    char out[1024];
    int pos = 0;
    for (int i = 0; i < n; ++i)
    {
        pos += snprintf(out + pos, sizeof(out) - pos, "%d ", arr[i]);
    }
    if (pos > 0)
    {
        out[pos - 1] = '\n';
    }
    else
    {
        out[pos++] = '\n';
    }
    write(STDOUT_FILENO, out, pos);
    free(arr);
}
else
{
    const char msg[] = "Unsupported command\n";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);
}
}
return 0;
}

```

## dynamic.c

```

#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include <dlfcn.h>

typedef float (*sin_integral_t)(float, float, float);
typedef int* (*sort_t)(int*, size_t);

int main()
{
    void* handle = dlopen("./libvar1.so", RTLD_LAZY);
    if (!handle)
    {
        write(2, "dlopen failed\n", 14);
        return 1;
    }

    sin_integral_t sin_integral = dlsym(handle, "sin_integral");
    sort_t sort = dlsym(handle, "sort");

    int current = 1;

```

```

char buf[512];

while (1)
{
    int n = read(0, buf, sizeof(buf) - 1);
    if (n <= 0)
    {
        break;
    }
    buf[n] = '\0';
    if (buf[n-1] == '\n')
    {
        buf[n-1] = '\0';
    }

    char* args[64];
    int argc = 0;
    char* p = buf;
    while (*p)
    {
        while (*p == ' ')
        {
            p++;
        }
        if (!*p)
        {
            break;
        }
        args[argc++] = p;
        while (*p && *p != ' ')
        {
            p++;
        }
        if (*p)
        {
            *p++ = '\0';
        }
    }

    if (argc == 0)
    {
        continue;
    }

    if (strcmp(args[0], "0") == 0)
    {
        dlclose(handle);
        if (current == 1)
        {
            handle = dlopen("./libvar2.so", RTLD_LAZY);
            current = 2;
        }
    }
}

```

```

else
{
    handle = dlopen("./libvar1.so", RTLD_LAZY);
    current = 1;
}
if (!handle)
{
    write(2, "dlopen failed\n", 14); return 1;
}
sin_integral = dlsym(handle, "sin_integral");
sort = dlsym(handle, "sort");
char msg[32];
int len = snprintf(msg, sizeof(msg), "Switched to var %d\n", current);
write(1, msg, len);
}
else if (strcmp(args[0], "1") == 0 && argc == 4)
{
    float a = strtod(args[1], NULL);
    float b = strtod(args[2], NULL);
    float e = strtod(args[3], NULL);
    float res = sin_integral(a, b, e);
    char out[64];
    int len = snprintf(out, sizeof(out), "%.6f\n", res);
    write(1, out, len);
}
else if (strcmp(args[0], "2") == 0 && argc >= 2)
{
    int n = (int)strtol(args[1], NULL, 10);
    if (n <= 0)
    {
        continue;
    }
    int* arr = malloc(n * sizeof(int));
    for (int i = 0; i < n && i + 2 < argc; ++i)
    {
        arr[i] = (int)strtol(args[2 + i], NULL, 10);
    }
    sort(arr, n);
    char out[1024];
    int pos = 0;
    for (int i = 0; i < n; ++i)
    {
        pos += snprintf(out + pos, sizeof(out) - pos, "%d ", arr[i]);
    }
    if (pos > 0)
    {
        out[pos - 1] = '\n';
    }
    else
    {
        out[pos++] = '\n';
    }
}

```



```

        write(STDOUT_FILENO, out, pos);
        free(arr);
    }
    else
    {
        const char msg[] = "Unsupported command\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    }
}

dlclose(handle);
return 0;
}

```

## Протокол работы программы

```

mrnek@WindowsLaptop:~/Projects/MAI/MAI_OS_LABS/LAB4$ LD_LIBRARY_PATH=. ./static1.out
1 1 2 0.00001
0.955284
2 10 10 9 8 7 6 5 4 3 2 1
1 2 3 4 5 6 7 8 9 10
mrnek@WindowsLaptop:~/Projects/MAI/MAI_OS_LABS/LAB4$ ./dynamic.out
1 0 1 0.000001
0.454152
2 5 1 2 5 4 3
1 2 3 4 5
0
Switched to var 2
1 0 2 0.0000001
1.389927
2 7 7 8 4 3 2 6 7
2 3 4 6 7 7 8

```

## Вывод

В ходе лабораторной работы я научился создавать динамические библиотеки, создавать программы, использующие динамические библиотеки и подключать их на этапе компиляции и во время выполнения программы.